



INSTITUTO FEDERAL DE EDUCAÇÃO, CIÊNCIA E TECNOLOGIA DE GOIÁS

PRÓ-REITORIA DE ENSINO

CÂMPUS GOIÂNIA

PÓS-GRADUAÇÃO LATO SENSU EM INTELIGÊNCIA ARTIFICIAL APLICADA

IGOR COUTINHO FERREIRA NOBRE

Projeto Final Integrado: Pipeline de Dados em Nuvem para Aprendizagem de Máquina

Reconhecimento de Atividade Humana com
Sensores de Smartwatch (WISDM)

Relatório de Disciplina

Professor: Dr. Sirlon Diniz

Disciplina: Projeto Final Integrado – Módulo 2 (Modelagem de Dados para IA, Aprendizagem de Máquina e Cloud Computing)

Coorientadores: Prof. Dr. Raphael Gomes e Prof. Dr. Otávio Calça

Goiânia, 2026.

Resumo

Este relatório documenta o projeto final integrado do Módulo 2 da Pós-Graduação em Inteligência Artificial Aplicada do Instituto Federal de Goiás (IFG), que reúne conteúdos das disciplinas de Modelagem de Dados para IA, Aprendizagem de Máquina e *Cloud Computing*. O problema escolhido é o reconhecimento automático de atividades humanas a partir de sinais de acelerômetro e giroscópio de *smartwatch*, com o objetivo de apoiar a decisão de um aplicativo de saúde e *fitness* sobre qual atividade um usuário está realizando em um dado intervalo de tempo. A solução implementa um *pipeline* completo de dados em arquitetura medalhão (bronze, prata e ouro), orquestrado pelo Apache Airflow, com extração de atributos estatísticos a partir dos sinais brutos do *dataset* WISDM, carga em nuvem (Amazon S3, AWS Glue Data Catalog e Amazon Athena) e transformação com dbt, incluindo testes de qualidade de dados. A tarefa de Aprendizagem de Máquina é uma classificação multiclasse (18 atividades), resolvida por uma rede neural *perceptron* multicamadas implementada manualmente com *NumPy* (*forward pass*, retropropagação e gradiente descendente com momento) e validada por comparação com o `MLPClassifier` do *scikit-learn*. O modelo supera com folga um *baseline* majoritário e atinge acurácia e F1-macro próximos a 0,68 no conjunto de teste, com análise de erros que identifica um agrupamento de atividades de alimentação/bebida fisicamente difíceis de distinguir apenas com sensores de pulso. Os resultados são publicados na camada ouro e consumidos por um painel de visualização, e a infraestrutura AWS utilizada é provisionada via CloudFormation. O relatório também descreve uma proposta de arquitetura equivalente inteiramente em serviços gerenciados da AWS e discute as limitações e os próximos passos da solução.

Palavras-chave aprendizagem de máquina; computação em nuvem; engenharia de dados; reconhecimento de atividade humana; *dbt*; *Airflow*.

1 INTRODUÇÃO

1.1 Contexto

Este relatório documenta o projeto final integrado do Módulo 2 da Pós-Graduação em Inteligência Artificial Aplicada do Instituto Federal de Goiás (Insti-

tuto Federal de Educação, Ciência e Tecnologia de Goiás (IFG)), trabalho interdisciplinar que reúne Modelagem de Dados para IA, Aprendizagem de Máquina e *Cloud Computing*. O enunciado do projeto interdisciplinar pede um *pipeline* completo de dados em nuvem, da ingestão à disponibilização de resultados para tomada de decisão: pelo menos uma fonte estruturada e uma não estruturada/semiestruturada; um *pipeline* de *Extract, Load, Transform* (ELT) com Airflow, dbt e uma base analítica; ao menos um serviço Amazon Web Services (AWS) (AMAZON WEB SERVICES, 2026); uma tarefa de Aprendizagem de Máquina implementada manualmente e com biblioteca, comparadas entre si; e um painel de visualização.

Escolhi o problema de reconhecimento de atividade humana (*Human Activity Recognition*) a partir de sinais de sensores de *smartwatch*, usando o *dataset* público WISDM (Weiss, 2019), por caber nesses requisitos dentro de um único domínio de aplicação, sem recorrer a fontes de dados desconhecidas só para preencher a lista de exigências.

1.2 Definição do problema e decisão apoiada

Domínio de aplicação: aplicativos de saúde e *fitness* que usam sensores de *smartwatch* (acelerômetro e giroscópio de pulso) para inferir o comportamento do usuário ao longo do dia.

Usuário/tomador de decisão: o próprio aplicativo (e, indiretamente, o usuário final e um profissional de saúde/educação física que acompanhe seus dados), que precisa decidir *qual rótulo de atividade atribuir* a uma janela de tempo do sinal do sensor.

Decisão apoiada: classificar automaticamente, a cada janela de 10 segundos de sinal, qual das 18 atividades do protocolo WISDM (andar, correr, subir escada, sentar, ficar em pé, digitar, escovar os dentes, comer/beber em suas variações, praticar esportes, escrever, bater palmas, dobrar roupa etc.) o usuário está realizando. Essa informação alimenta contadores de atividade, alertas de sedentarismo e relatórios de rotina, sem exigir que o usuário registre manualmente o que está fazendo.

Fontes de dados: o *dataset* WISDM, que chega em duas naturezas já na camada bronze: sinais brutos de acelerômetro e giroscópio (semiestruturados) e arquivos ARFF com atributos já calculados pelos autores do *dataset* (estruturados), descritas em detalhe na Seção 2.1. O modelo de Aprendizagem de Máquina é treinado sobre uma tabela estruturada equivalente, recalculada pelo próprio *pipeline* a partir dos sinais brutos (Seção 3.2).

Tarefa de Aprendizagem de Máquina: classificação multiclasse supervisionada (18 classes balanceadas), detalhada no Capítulo 4.

Resultado esperado da solução: uma tabela de predições e métricas de desempenho por atividade, publicada na camada ouro do *pipeline* e consumida por um painel de visualização. Com ela dá para comparar o desempenho do modelo entre atividades e sujeitos e decidir que tipo de instrumentação adicional (por exemplo, sensor no telefone além do *smartwatch*) valeria a pena para melhorar a distinção de atividades mais difíceis.

1.3 Objetivos

O objetivo geral foi construir, de ponta a ponta, um *pipeline* de dados em nuvem da ingestão de dados brutos até a disponibilização de previsões de Aprendizagem de Máquina em um painel de decisão. Os objetivos específicos foram:

1. identificar, coletar e documentar uma fonte de dados estruturados e uma semiestruturada adequadas ao problema (Seção 2.1);
2. construir um *pipeline* reproduzível de ingestão, tratamento, extração de atributos, carga e disponibilização dos dados (Capítulo 3);
3. modelar os dados analíticos em camadas (*staging*, *marts* e *gold*) usando dbt, com testes e documentação (Seção 3.3);
4. definir, preparar os dados e treinar/avaliar um modelo de Aprendizagem de Máquina implementado manualmente e com biblioteca, comparando os dois (Capítulo 4);
5. usar serviços da AWS para armazenamento, catalogação e consulta dos dados, com a infraestrutura provisionada via *CloudFormation* (Seção 2.3);
6. disponibilizar os resultados em um painel de visualização que apoie a decisão descrita no Seção 1.2 (Capítulo 5);
7. registrar as limitações da solução e propor uma arquitetura equivalente inteiramente em serviços gerenciados da AWS (Seção 2.4 e Capítulo 6).

Os capítulos seguintes seguem essa mesma ordem: dados e arquitetura (Capítulo 2), *pipeline* de ELT (Capítulo 3), Aprendizagem de Máquina (Capítulo 4), visualização e reprodutibilidade (Capítulo 5) e conclusão (Capítulo 6).

2 DADOS E ARQUITETURA DA SOLUÇÃO

2.1 Conjuntos de dados

Foi utilizada uma única base de dados de origem, o *WISDM Smartphone and Smartwatch Activity and Biometrics Dataset* (Weiss, 2019), disponibilizado no repositório da UCI Machine Learning. A DAG de ingestão (Seção 3.1) baixa o *dataset completo* via `kagglehub.dataset_download` e envia todo o conteúdo, sem filtrar nada, para a camada bronze do S3 (`raw/wisdm/`): 896 MiB em 464 arquivos. Esse *dump* já contém as duas naturezas de dado que a Seção 4.2 do enunciado pede, em dois diretórios do *dataset* original:

- **Estruturado:** `arff_files/` (200 arquivos). O *dataset* já vem pronto em formato ARFF (*Attribute-Relation File Format*, do WEKA), que possui formato tabular, uma linha por janela de 10s com os atributos estatísticos já calculados pelos próprios autores do WISDM (distribuição em *bins*, média, pico, desvio absoluto médio, desvio padrão, variância etc.);
- **Semiestruturado:** `raw/` (204 arquivos: 51 sujeitos $\times$ 4 combinações *sensor/dispositivo*). São os sinais brutos de acelerômetro e giroscópio, coletados simultaneamente por *smartphone* e *smartwatch* a aproximadamente 20 Hz, no formato `data_<sujeito>_<sensor>_<dispositivo>.txt`, uma linha por

amostra. O enunciado lista explicitamente “dados de sensores” entre os exemplos válidos dessa categoria: cada linha tem formato regular, mas o arquivo não é diretamente tabular para análise (mistura as 18 atividades em sequência, taxa de amostragem apenas nominal, linhas malformadas).

Os dois diretórios chegam juntos no mesmo *download*, satisfazendo a exigência de requisitos do projeto integrado onde se pede (um conjunto estruturado e outro semiestruturado) já na origem.

O restante do *pipeline* (Seção 3.2), no entanto, consome apenas o **raw/**: que implementa o “processamento e extração de atributos” a partir do dado bruto, não que reaproveite um resultado já processado por terceiros. Por isso o **arff_files/** fica armazenado na bronze, mas não entra no *pipeline*: o dado estruturado que alimenta a camada prata e o modelo de Aprendizagem de Máquina (Capítulo 4) é a tabela **wisdm_features**, recalculada do zero: sincronização de acelerômetro e giroscópio, as janelas de 10 segundos e cálculo de estatísticas por janela (24 atributos numéricos + rótulo de atividade). Foi processado só o dispositivo *watch* nesta primeira versão, já que o problema descrito no Seção 1.2 é sobre um aplicativo baseado em *smartwatch*; os arquivos de *phone* ficam na bronze, mas sem uso corrente pelo restante do *pipeline*.

A Tabela 1 usa a nomenclatura de camadas da arquitetura medalhão (Seção 2.2) para localizar cada formato de dado no *pipeline*.

Tabela 1 – *Resumo dos conjuntos de dados utilizados.*

Camada	Tipo	Formato / local	Conteúdo
Bronze	Estruturado + Semiestruturado	S3 raw/wisdm/	≈ 896 MiB, 464 arquivos: arff_files/ (200, estruturado) e raw/ (204, semiestruturado, sinais brutos)
Silver	Estruturado	Parquet part. por sujeito, S3 silver/wisdm_features/	16.721 janelas de 10s do <i>watch</i> , 24 atributos + rótulo, recalculados a partir do raw/
Gold	Estruturado	Tabelas Athena/Glue	Agregados por sujeito e por atividade, e resultados do modelo de ML

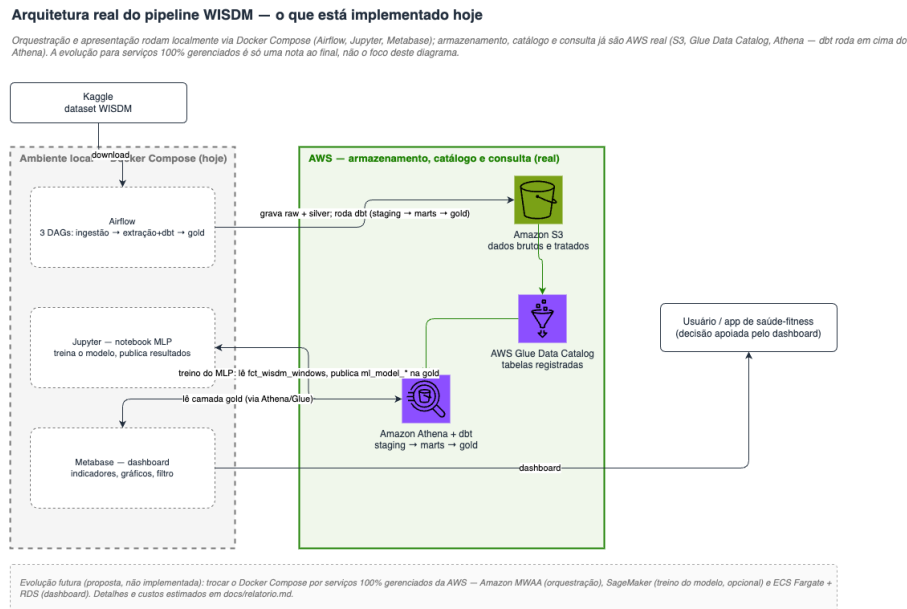
Fonte: Elaborada pelo autor, a partir de (Weiss, 2019) e da execução do *pipeline* descrito no Capítulo 3.

Os dados foram utilizados para treino, validação e teste do modelo de Aprendizagem de Máquina (Capítulo 4): 51 sujeitos, com um protocolo de coleta balanceado (cerca de 3 minutos por atividade e sujeito), resultam nas 16.721 janelas válidas usadas no Capítulo 4. A origem, o formato, os campos e as limitações de cada camada estão documentados nos arquivos **wisdm_silver/schema.py** e **wisdm_silver/feature_extraction.py** do repositório, além dos testes de dados descritos no Seção 3.3.

2.2 Arquitetura implementada

Segui uma arquitetura em camadas (*medalhão*): bronze (dados brutos), prata (dados tratados e tabulares) e ouro (agregados de negócio e resultados do modelo). A Figura 1 mostra o que está de fato implementado e rodando. A orquestração (Airflow) e a camada de apresentação (Jupyter para o treino do modelo e o painel de visualização) rodam localmente via Docker e foi utilizado o *Docker Compose* para orquestrá-los; já o armazenamento, o catálogo de dados e a consulta analítica são serviços reais da AWS: Amazon S3, AWS Glue Data Catalog e Amazon Athena, com o dbt rodando as transformações em cima do Athena.

Figura 1 — *Arquitetura implementada do pipeline WISDM: orquestração e apresentação locais, armazenamento/catálogo/consulta em nuvem real.*



Fonte: Elaborada pelo autor.

Essa divisão foi baseada no escopo e limitações de infraestrutura: como o ambiente de nuvem é uma *sandbox* do AWS Academy Lab (Seção 2.5), com credenciais temporárias e sem acesso a Cost Explorer/Billing, deixei localmente os componentes que preciso ter disponíveis o tempo todo para desenvolver e depurar (orquestrador, notebook, painel). Os componentes de dados, foco de avaliação do enunciado, esses já são inteiramente providos pela AWS.

2.3 Uso de recursos da AWS

Os recursos da AWS (AMAZON WEB SERVICES, 2026) utilizados, por ordem de utilização: primeiro o CloudFormation, que provisiona tudo o mais antes de qualquer DAG rodar.

- **AWS CloudFormation:** toda a infraestrutura da solução (*bucket* S3, política de acesso público bloqueado, criptografia *server-side*, regra de ciclo de vida e

os dois bancos Glue) é definida como código em `infra/cloudformation.yaml` e provisionada com `infra/deploy_infra.sh` antes da primeira execução de qualquer DAG, evitando que a infraestrutura seja criada implicitamente como efeito colateral do *pipeline*;

- **Amazon S3:** um único *bucket* organizado em três prefixos: `raw/wisdm/` (bronze), `silver/wisdm_features/` (prata, particionado por `subject`) e `athena-results/` (resultados/metadados de consulta do Athena, com expiração automática de 7 dias via regra de ciclo de vida do *bucket*);
- **AWS Glue Data Catalog:** dois bancos de dados, um por camada (`projeto_integrado_wisdm_silver` e `projeto_integrado_wisdm_gold`), usados como substituto do Snowflake neste ambiente de laboratório (decisão de escopo autorizada previamente), já que o Athena consulta diretamente os arquivos no S3 através das tabelas externas registradas no Glue;
- **Amazon Athena:** mecanismo de consulta *serverless* usado pelo dbt (via *adapter dbt-athena*) para materializar as *views*/tabelas de *staging*, *marts* e *gold* descritas no Seção 3.3.

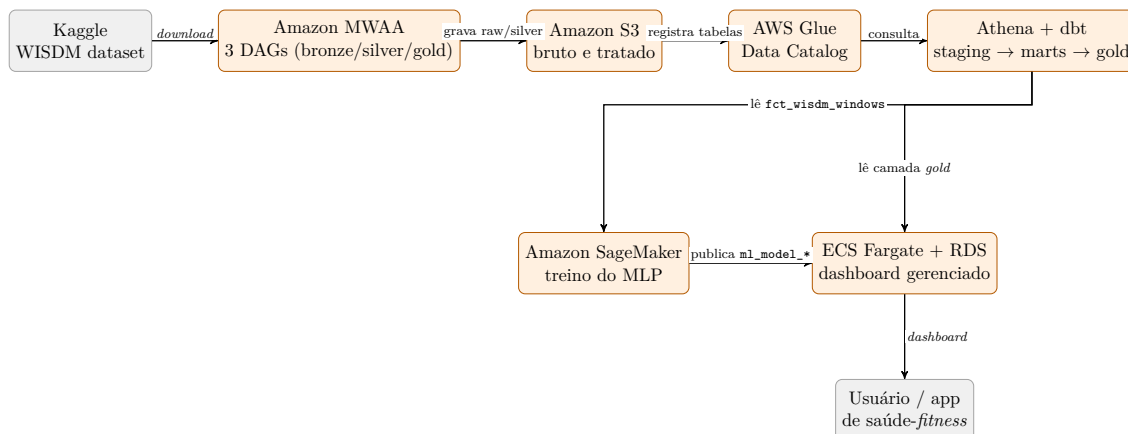
O recurso `WisdmDataBucket` do template (`infra/cloudformation.yaml`, disponível no repositório do projeto (Nobre, 2026)) define o *bucket* S3 com bloqueio de acesso público e criptografia *server-side*. O mesmo template também declara os dois bancos do Glue Data Catalog citados acima, um por camada da medalhão, separados para manter cada camada no seu próprio *schema* do catálogo, evitando colisão de nomes de tabela. Os bancos ficam visíveis e consultáveis no Athena assim que o CloudFormation é aplicado, antes mesmo de qualquer DAG rodar.

2.4 Proposta de arquitetura equivalente 100% AWS

Na Figura 1, a orquestração, o treino do modelo e o painel ficaram rodando localmente por restrição prática do ambiente de laboratório e limitação de recursos, não por limitação da arquitetura em si. Por isso como proposta, na Figura 2, uma solução equivalente inteiramente em serviços gerenciados da AWS, trocando cada componente local pelo seu equivalente em nuvem sem alterar o formato dos dados ou a lógica de transformação:

- **Amazon MWAA** (*Managed Workflows for Apache Airflow*) no lugar do Airflow em *Docker Compose*, executando as mesmas três DAGs (`kaggle_wisdm_to_s3`, `wisdm_bronze_to_silver` e `wisdm_silver_to_gold`);
- **Amazon SageMaker** para o treino do MLP (Capítulo 4) no lugar do Jupyter local, lendo `fct_wisdm_windows` diretamente do Athena;
- **Amazon ECS Fargate** com um banco gerenciado (**Amazon RDS**) no lugar do Metabase em contêiner local, servindo o mesmo painel de visualização (Capítulo 5) de forma continuamente disponível.

Figura 2 – *Proposta de arquitetura equivalente 100% AWS: os mesmos componentes lógicos da Figura 1, substituindo Docker Compose por serviços gerenciados.*



Fonte: Elaborada pelo autor.

Não cheguei a implementar essa migração porque o ambiente disponível é uma conta *sandbox* do AWS Academy Learner e devido às limitações de recursos, com permissões de IAM restritas e sem garantia de continuidade entre sessões (Seção 2.5), condições que não combinam com MWAA, SageMaker e Fargate, que exigem papéis de IAM mais amplos e ficam ativos (e cobrando) continuamente. Deixo a proposta registrada como caminho de evolução natural, retomado no Capítulo 6.

2.5 Registro de custos

O projeto roda em uma conta AWS Academy Lab, que não expõe Cost Explorer nem Billing ao aluno, então não tenho fatura para citar. O registro a seguir é uma estimativa feita com a AWS Pricing Calculator (tarifas públicas de *us-east-1*), aplicada sobre o uso real medido no repositório.

O valor fica dentro do *free tier* da maioria das contas AWS novas e é consumido como créditos do laboratório, sem custo real. Já para a arquitetura 100% AWS proposta na Seção 2.4, o custo deixaria de ser marginal: MWAA, SageMaker e Fargate cobram por disponibilidade contínua, não por uso pontual como Athena/S3/Glue, uma das razões práticas pelas quais não implementei essa versão neste projeto de disciplina.

3 PIPELINE DE DADOS: PROCESSAMENTO, EXTRAÇÃO DE ATRIBUTOS E ELT

O *pipeline* em três etapas orquestradas pelo Apache Airflow (APACHE SOFTWARE FOUNDATION, 2024), ferramenta de código aberto que agenda e

Tabela 2 – *Estimativa de custo mensal da solução implementada (S3 + Glue + Athena).*

Serviço	Uso medido/estimado	Preço unitário	Custo/mês
S3	≈ 0,88 GB armazenados + 1.030 PUT/COPY/POST/-LIST + 204 GET	\$0,023/GB-mês + \$0,005–0,0004/mil req.	\$0,02
Athena	≈ 200 <i>queries</i> (dbt run/test + desenvolvimento), ≈ 2 GB escaneados	\$5/TB escaneado	\$0,01
Glue Data Catalog	2 bancos, <10 tabelas, <1.000 acessos	1º milhão de objetos/acessos grátis	\$0,00
CloudFormation	Provisionamento do S3 + Glue	Sem custo pelo serviço em si	\$0,00
Total estimado			≈\$0,04

Fonte: Elaborada pelo autor, com base em `arquitetura/registro_de_custos.md` e nas tarifas públicas da AWS para `us-east-1`.

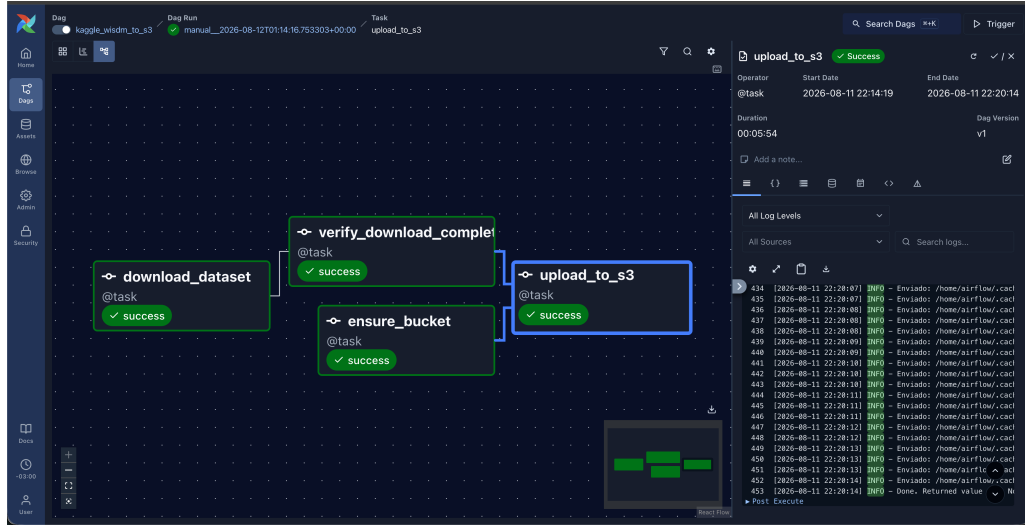
monitora fluxos de tarefas encadeadas, cada etapa em uma DAG (*Directed Acyclic Graph* (DAG)) própria: ingestão (bronze), extração de atributos e carga tabular (prata) e agregação de negócio (ouro). Encadeei as DAGs por dependência de dados (*Airflow Assets*), não por horário, para que a seguinte só dispare quando a anterior termina com sucesso, evitando reprocessar a camada ouro sobre uma prata desatualizada.

3.1 Camada bronze: ingestão

A DAG `kaggle_wisdm_to_s3` garante que o *bucket* S3 existe, baixa o *dataset* completo do Kaggle com `kagglehub`, verifica que os quatro arquivos brutos esperados (`phone_accel`, `phone_gyro`, `watch_accel`, `watch_gyro`) estão presentes para todos os 51 sujeitos e só então envia os arquivos ao S3 (Figura 3). Coloquei essa verificação de completude antes do envio porque já tive um problema parecido em um projeto anterior do mesmo grupo de disciplinas (*dataset* WESAD): um *download* parcial avançou silenciosamente pelas camadas seguintes com apenas parte dos sujeitos.

A DAG é definida (`kaggle_wisdm_to_s3.py`, disponível no repositório do projeto (Nobre, 2026)) no estilo *TaskFlow API* do Airflow: cada função decorada com `@task` vira uma tarefa, e o operador `»` declara a dependência de execução. Usei o parâmetro `outlets=[BRONZE_S3_ASSET]` na tarefa `upload_to_s3` para disparar automaticamente a DAG da camada prata (Seção 3.2) assim que esta termina com sucesso, o mesmo mecanismo de *Airflow Assets* se repete, com `schedule/outlets` trocados, no encadeamento prata → ouro (Seção 3.2 e Seção 3.4).

Figura 3 – Execução bem-sucedida da DAG *kaggle_wisdm_to_s3* (camada bronze).



Fonte: Elaborada pelo autor.

3.2 Extração de atributos: camada prata

A DAG *wisdm_bronze_to_silver* lê, para cada sujeito, os arquivos *watch_accel* e *watch_gyro* da camada bronze e aplica o processamento a seguir (implementado em *wisdm_silver/feature_extraction.py*):

1. **Sincronização:** acelerômetro e giroscópio não amostram exatamente nos mesmos instantes, então cada amostra do acelerômetro é casada com a amostra mais próxima do giroscópio (*pandas.merge_asof*, direção *nearest*), com uma tolerância calculada como metade do intervalo mediano entre amostras consecutivas do próprio acelerômetro, em vez de um valor fixo em milissegundos, já que a documentação pública do *dataset* não confirma a unidade exata do *timestamp*. Amostras sem par dentro da tolerância são descartadas e a contagem é registrada em *log*;
2. **Janela:** o sinal sincronizado é dividido em janelas fixas e disjuntas de 10 segundos (200 amostras a 20 Hz), sem sobreposição, cobrindo todas as 18 atividades gravadas na ordem do arquivo;
3. **Filtro de pureza:** uma janela só vira uma linha da tabela se pelo menos 90% das amostras dentro dela pertencerem à mesma atividade (a atividade majoritária vira o rótulo), descartando janelas de transição entre atividades;
4. **Extração de atributos:** para cada um dos 6 canais (*accel_x/y/z*, *gyro_x/y/z*), são calculadas 4 estatísticas (média, desvio padrão, mínimo e máximo), totalizando 24 atributos numéricos por janela.

O resultado é gravado em Parquet, particionado por sujeito, na camada prata do S3, e uma tabela externa *wisdm_features* é registrada (ou atualizada com novas partições) no Glue Data Catalog, para consulta via Athena. A DAG só avança para o dbt depois de confirmar que todos os 51 sujeitos geraram janelas válidas (evidência de execução na Figura 4, junto com a camada ouro).

Essa DAG é disparada por `schedule=[BRONZE_S3_ASSET]`, o outro lado do encadeamento citado no Seção 3.1: o Airflow a dispara automaticamente quando o *asset* `BRONZE_S3_ASSET` é declarado como `outlets` na tarefa `upload_to_s3`, uma dependência de **dado**, não de tempo. Prefiro esse mecanismo a um horário fixo ou a um `TriggerDagRunOperator` explícito. A tarefa `run_dbt`, por sua vez, declara `outlets=[SILVER_DBT_ASSET]`, repetindo o mesmo mecanismo para disparar a DAG da camada ouro (Seção 3.4).

3.3 Modelagem com dbt: staging, marts e gold

Utilizei o dbt (DBT LABS, 2024) para a transformação tabular, ferramenta que deixa escrever a camada de transformação em SQL versionado, testável e documentado, em vez de *scripts* soltos, usando o *adapter dbt-athena* para materializar os *models* diretamente sobre as tabelas do Glue Data Catalog. Organizei os *models* em três níveis, seguindo a convenção de camadas do próprio dbt e dando continuidade à arquitetura medalhão do projeto (Seção 2.2); dentro do nível *marts*, apliquei a modelagem dimensional de Kimball e Ross (Kimball; Ross, 2013), que organiza os dados em tabelas fato (eventos/medições) e dimensão (atributos descritivos):

staging (`stg_wisdm_features`, materializado como *view*): renomeia colunas da fonte externa e cria uma chave substituta `window_uid` (`subject_id || activity_label || window_id`);

marts (`dim_subject` e `fct_wisdm_windows`, materializados como tabela): `fct_wisdm_windows` é a tabela fato no grão de janela de sinal, usada tanto pelo treino/avaliação do modelo (Capítulo 4) quanto como base da camada seguinte; `dim_subject` resume, por sujeito, o total de janelas válidas e a pureza média de rótulo;

gold (`gold_subject_activity_summary` e `gold_activity_benchmark`): agregados de negócio no grão sujeito $\times$ atividade e no grão de atividade (benchmark populacional), prontos para consumo direto pelo painel de visualização (Capítulo 5).

O *model* de *staging* (`stg_wisdm_features.sql`, disponível no repositório do projeto (Nobre, 2026)), primeira dag da transformação, lê a fonte externa (`source`, a tabela `wisdm_features` registrada no Glue pela DAG da prata), renomeia colunas e monta a chave substituta `window_uid`, usada nos testes de unicidade e propagada por todos os *models* seguintes. Do outro lado da cadeia, o *model* de *gold* `gold_subject_activity_summary` (construído sobre o mart `fct_wisdm_windows`, um `select *` direto sobre o *staging*) calcula a participação de cada atividade na sessão do sujeito e a magnitude do vetor de médias dos sinais como *proxy* de intensidade de movimento.

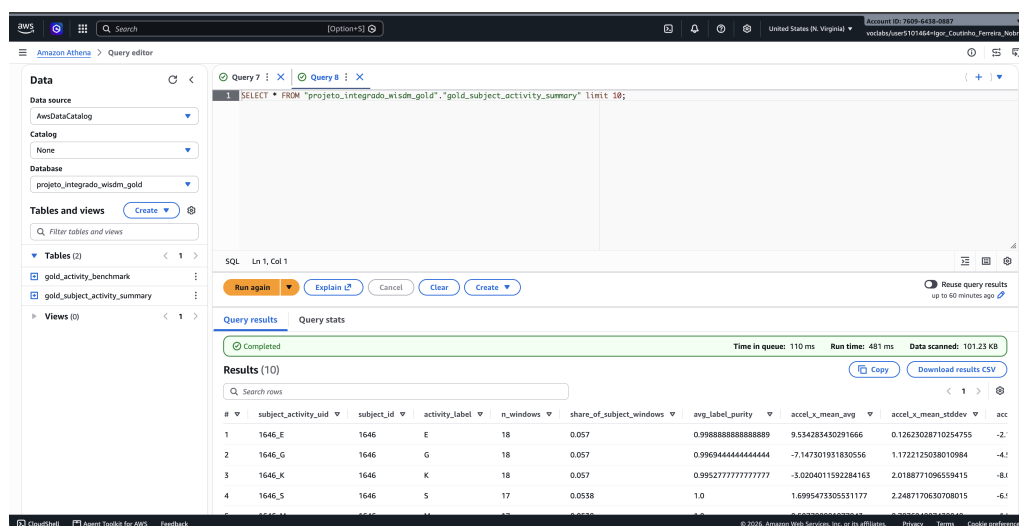
Escrevi 23 testes dbt (`unique`, `not_null` e `accepted_values`) distribuídos pelos cinco *models* para verificar a qualidade dos dados: por exemplo, unicidade de `window_uid` e `subject_activity_uid`, ausência de nulos nas chaves e no rótulo de atividade, e validação de que `activity_label` pertence ao conjunto das 18 letras válidas do protocolo WISDM. Documentei a descrição e as colunas de cada *model* nos arquivos `_stg_wisdm_features.yml`, `_marts.yml` e `_gold.yml`. Os testes

rodam automaticamente após cada `dbt run`, como parte das mesmas DAGs (etapas `run_dbt` e `run_dbt_gold`, evidência de execução na Figura 4).

3.4 Camada ouro e orquestração final

A DAG `wisdm_silver_to_gold` confirma que os 51 sujeitos esperados estão registrados como partições da tabela prata antes de materializar os *models* de ouro, para não gastar tempo (nem consulta Athena) agregando sobre uma prata incompleta. Ela é disparada automaticamente pelo mesmo mecanismo de *Airflow Assets* usado entre as camadas bronze e prata (`schedule=[SILVER_DBT_ASSET]`), assim que a etapa `run_dbt` da DAG anterior termina com sucesso (Figura 4), fechando a cadeia bronze → prata → ouro inteiramente por dependência de dado, sem agendamento por horário em nenhuma das três DAGs.

Figura 4 – *Execução bem-sucedida da DAG wisdm_silver_to_gold, incluindo os 12 testes dbt da camada gold.*



Fonte: Elaborada pelo autor.

Escolhi esse encadeamento por dependência de dados, em vez de horário fixo pois com o sandbox precisa ser iniciado toda vez que vai iniciar, para garantir a rastreabilidade entre dado bruto, dado tratado e predição pedida na avaliação do módulo: cada camada só avança quando a anterior confirma que produziu o conjunto completo de dados esperado.

4 APRENDIZAGEM DE MÁQUINA

Esta seção documenta o notebook `ml/mlp_activity_classification.ipynb`, onde consumo a tabela `fct_wisdm_windows` (Seção 3.3) e implemento, treino e avalio o modelo de Aprendizagem de Máquina da solução. O núcleo deste capítulo é a implementação manual do MLP, do zero, em *NumPy*; uma segunda implementação,

feita com biblioteca, aparece apenas como linha de validação na tabela de resultados (Tabela 3).

4.1 Definição da tarefa

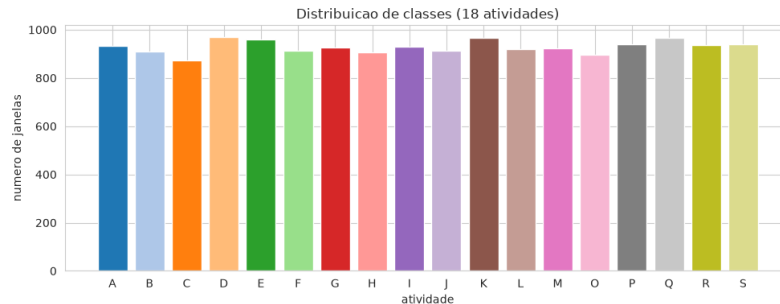
A tarefa é uma **classificação multiclasse supervisionada**: dada uma janela de 10 segundos de sinal (24 atributos estatísticos de acelerômetro e giroscópio, descritos no Seção 3.2), prever qual das 18 atividades do protocolo WISDM foi realizada. Das 24 colunas do `mart`, `window_uid` e `window_id` são apenas identificadores, e deixei `subject_id` de fora das *features* de propósito: se entrasse como atributo, o modelo poderia aprender a identidade do sujeito em vez do padrão de movimento da atividade, inflando a métrica de forma enganosa.

4.2 Análise exploratória

Antes de treinar qualquer modelo, inspecionei os dados quanto a valores ausentes/infinitos, balanceamento de classes, cobertura por sujeito, escala das *features* e separabilidade visual entre atividades. Não encontrei valores nulos, infinitos ou rótulos ausentes.

As 18 classes estão bem balanceadas (Figura 5), consequência do protocolo de coleta original (cerca de 3 minutos por atividade e sujeito). Também conferi a cobertura por sujeito: de $51 \times 18 = 918$ combinações possíveis de sujeito e atividade, 11 (1,2%) não têm nenhuma janela válida, uma taxa de lacunas aceitável para o *split* da Seção 4.3.

Figura 5 – Distribuição do número de janelas válidas por atividade (18 classes).



Fonte: Elaborada pelo autor.

A *feature* em detalhe (`accel_y_std`, desvio padrão do eixo Y do acelerômetro, uma *proxy* de quanto o pulso balançou na janela): atividades paradas ou sentadas (*sitting*, *standing*, *typing*) concentram-se perto de zero, enquanto *jogging* e *walking* têm variação bem maior, o que é fisicamente coerente. Uma redução por *Principal Component Analysis* (PCA) a 2 dimensões, usada apenas para visualização, confirma o mesmo padrão: atividades de movimento amplo formam grupos visualmente separados, enquanto atividades paradas ou sentadas ficam mais próximas entre si. Esse padrão reaparece, com números em vez de intuição visual, na análise de erros do Seção 4.7.

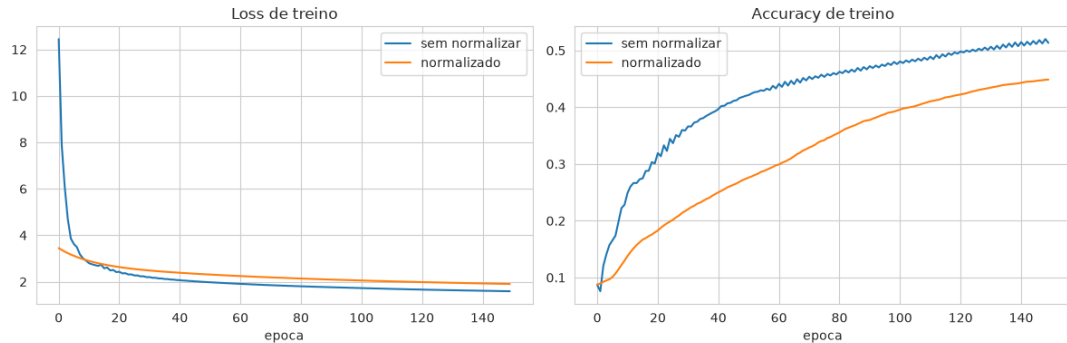
4.3 Preparação dos dados

Foi feito o *split* treino/validação/teste **por sujeito**, não por linha: dos 51 sujeitos, 35 para treino, 8 para validação e 8 para teste, garantindo que nenhum sujeito apareça em mais de um conjunto. Um *split* aleatório por linha enviesaria o resultado, já que janelas consecutivas do mesmo sujeito na mesma atividade são muito parecidas: o modelo aprenderia a reconhecer o sujeito, não o padrão geral da atividade. Conferi o balanceamento de classes depois do *split* e os três conjuntos mantêm proporções semelhantes.

Como `accel_*` e `gyro_*` estão em escalas fisicamente diferentes (m/s^2 contra rad/s), apliquei normalização *z-score*, $x' = (x - \mu)/\sigma$, calculando μ e σ **apenas com o conjunto de treino**; caso contrário, informação da validação e do teste vazaria para o pré-processamento do treino.

Treinei o mesmo MLP (arquitetura fixa, 150 épocas, sem parada antecipada, mesma *seed*) uma vez com os dados brutos e outra com os dados normalizados. O resultado (Figura 6) foi o oposto do esperado: ao final das 150 épocas, a *accuracy* de treino chegou a 0,513 sem normalizar contra 0,449 normalizado. As *features* brutas possuem magnitude maior que 1, então o gradiente também sai maior nessa inicialização específica, funcionando como se a taxa de aprendizado efetiva fosse mais alta, e não porque a normalização atrapalhe por si só.

Figura 6 – *Convergência do treino com e sem normalização z-score (150 épocas, mesma seed).*



Fonte: Elaborada pelo autor.

Esse teste não invalidou a normalização, mas mudou o motivo pelo qual mantive-a: as unidades físicas diferentes dos dois sensores e a regularização L2 usada na busca de hiperparâmetros (Seção 4.5), que penaliza a magnitude dos pesos igualmente para todas as *features* e só é justa quando elas estão na mesma escala, e não uma suposta convergência mais rápida, que os números acima não confirmaram.

4.4 Baseline

Usei como *baseline* o classificador majoritário (sempre prevê a classe mais frequente no treino). No conjunto de teste, ele atinge acurácia de 0,056 e F1-macro de 0,006, próximo do esperado para um chute aleatório entre 18 classes equilibradas

($1/18 \approx 0,056$). Isso confirma que o *dataset* não tem uma classe dominante capaz de inflar artificialmente essa métrica.

4.5 MLP implementado do zero

O núcleo do trabalho é uma rede *perceptron* multicamadas (*Multilayer Perceptron* (MLP)) que implementei em *NumPy* (Harris *et al.*, 2020) puro, a biblioteca de computação numérica com vetores e matrizes que dá a base das operações de álgebra linear usadas aqui, sem usar `sklearn.neural_network`, justamente para expor o *forward pass*, a retropropagação e o cálculo do gradiente. A arquitetura é simples: entrada (24 atributos) $\rightarrow$ uma camada escondida com *Rectified Linear Unit* (ReLU) $\rightarrow$ saída (18 classes) com *softmax*:

$$Z_1 = XW_1 + b_1, \quad A_1 = \text{ReLU}(Z_1) \quad (1)$$

$$Z_2 = A_1W_2 + b_2, \quad \hat{Y} = \text{softmax}(Z_2) \quad (2)$$

com $\text{softmax}(z)_k = e^{z_k} / \sum_j e^{z_j}$ e função de custo de entropia cruzada. Inicializo os pesos W_1 e W_2 com o esquema de He (normal, desvio padrão $\sqrt{2/n_{\text{entrada}}}$), recomendado quando a ativação é ReLU pois evita começar com ativações todas próximas de zero; os *biases* começam em zero. O construtor também guarda os hiperparâmetros de treino (*learning_rate*, *momentum*, *l2_lambda*) e o estado de velocidade usado pelo gradiente descendente com *momentum* (Subseção 4.5.3).

4.5.1 Construção incremental e verificação das peças

Antes de montar a classe `MLPHardcode` (`mlp_activity_classification.ipynb`, disponível no repositório do projeto (Nobre, 2026)), testei isoladamente cada peça (*ReLU*, *softmax*, entropia cruzada) e depois um teste com dado sintético (200 amostras, sem padrão real): em 50 épocas, o *loss* caiu de 3,800 para 1,482, sinal de que o *forward/backward* não tinha erro grosseiro antes de treinar nos dados reais.

Código 1 `mlp_activity_classification.ipynb`: inicialização de pesos (esquema de He) na classe `MLPHardcode`

```

1 def __init__(self, n_input, n_hidden, n_output, learning_rate=0.05, momentum=0.9,
2               l2_lambda=0.0, seed=42):
3     rng = np.random.default_rng(seed)
4     # inicializacao He (variancia 2/n_entradas) - e a recomendada quando a
5     # ativacao e ReLU, evita começar com ativacoes todas proximas de zero
6     self.W1 = rng.normal(0, np.sqrt(2.0 / n_input), size=(n_input, n_hidden))
7     self.b1 = np.zeros((1, n_hidden))
8     self.W2 = rng.normal(0, np.sqrt(2.0 / n_hidden), size=(n_hidden, n_output))
9     self.b2 = np.zeros((1, n_output))

```

A inicialização dos pesos Código 1 mostra o desvio padrão de cada peso ($\sqrt{2/n_{\text{entrada}}}$) escala com o número de entradas da camada: sem esse ajuste, uma camada com muitas entradas somaria muitos termos pequenos e produziria ativações artificialmente grandes ou pequenas antes mesmo do primeiro passo de treino. Os *biases* começam em zero, prática padrão que não interfere na quebra de simetria (que já vem dos pesos aleatórios).

4.5.2 Forward pass e retropropagação

O Código 2 mostra, disponível no repositório do projeto (Nobre, 2026), o *forward pass* e a retropropagação, correspondentes diretamente às Equações 1 e 2: **forward** calcula Z_1 , A_1 , Z_2 e $\hat{Y}$; **backward** deriva o gradiente de cada peso a partir do erro de saída ($\hat{Y} - Y$, simplificado pela combinação *softmax* + entropia cruzada), propagando-o de volta pela camada escondida com a derivada da ReLU, e somando o termo de regularização L2 ao gradiente de cada peso.

Código 2 mlp_activity_classification.ipynb: forward pass e retropropagação de MLPHardcode

```

1 def forward(self, X: np.ndarray) -> dict:
2     Z1 = X @ self.W1 + self.b1
3     A1 = relu(Z1)
4     Z2 = A1 @ self.W2 + self.b2
5     A2 = softmax(Z2)
6     return {"Z1": Z1, "A1": A1, "Z2": Z2, "A2": A2}
7
8 def backward(self, X: np.ndarray, y_onehot: np.ndarray, cache: dict) -> dict:
9     n = X.shape[0]
10    # softmax + entropia cruzada juntos simplificam pra isso
11    dZ2 = (cache["A2"] - y_onehot) / n
12    dW2 = cache["A1"].T @ dZ2 + self.l2_lambda * self.W2
13    db2 = dZ2.sum(axis=0, keepdims=True)
14
15    dA1 = dZ2 @ self.W2.T
16    dZ1 = dA1 * relu_derivative(cache["Z1"])
17    dW1 = X.T @ dZ1 + self.l2_lambda * self.W1
18    db1 = dZ1.sum(axis=0, keepdims=True)
19
20    return {"W1": dW1, "b1": db1, "W2": dW2, "b2": db2}

```

4.5.3 Laço de treino e parada antecipada

A cada época, calculo *forward*, *backward* (Código 2) e atualizo os pesos. Em vez do gradiente descendente clássico ($W \leftarrow W - \eta \nabla W$), uso *momentum*: a velocidade de cada parâmetro acumula uma fração (0,9) da velocidade da época anterior ($v \leftarrow \text{momentum} \cdot v - \eta \nabla W$, $W \leftarrow W + v$), suavizando a trajetória de otimização e ajudando a atravessar mínimos locais rasos. O Código 3 mostra o método **fit** completo, que orquestra esse laço.

Código 3 mlp_activity_classification.ipynb: método fit de MLPHardcode (laço de treino e parada antecipada)

```

1 def fit(self, X_train, y_train, epochs=300, X_val=None, y_val=None, verbose_every=0,
2     patience=None, restore_best_weights=True):
3     y_onehot = one_hot(y_train, self.n_output)
4     history = {"train_loss": [], "train_accuracy": [], "val_loss": [], "val_accuracy": []}
5
6     best_val_loss = np.inf
7     best_state = None
8     best_epoch = 0
9     epochs_without_improvement = 0
10
11     for epoch in range(1, epochs + 1):
12         cache = self.forward(X_train)
13         grads = self.backward(X_train, y_onehot, cache)
14         self._apply_gradients(grads)
15
16         train_loss = cross_entropy(cache["A2"], y_onehot) + self._l2_penalty() / X_train.
17             shape[0]
18         history["train_loss"].append(train_loss)
19         history["train_accuracy"].append((cache["A2"].argmax(axis=1) == y_train).mean())
20
21         if X_val is not None and y_val is not None:
22             val_probs = self.forward(X_val)
23             val_loss = cross_entropy(val_probs, one_hot(y_val, self.n_output))
24             history["val_loss"].append(val_loss)
25             history["val_accuracy"].append((val_probs.argmax(axis=1) == y_val).mean())
26
27         # early stopping: se o val_loss parou de melhorar, corta o treino
28         # e volta os pesos pra epoca em que ele foi melhor (senao o modelo
29         # comeca a decorar o treino em vez de generalizar)
30         if val_loss < best_val_loss - 1e-6:
31             best_val_loss = val_loss
32             best_epoch = epoch
33             epochs_without_improvement = 0
34             if restore_best_weights:
35                 best_state = {k: v.copy() for k, v in self._params().items()}
36         else:
37             epochs_without_improvement += 1
38
39         if patience is not None and epochs_without_improvement >= patience:
40             break
41
42         if restore_best_weights and best_state is not None:
43             params = self._params()
44             for name in params:
45                 params[name][...] = best_state[name]
46
47         history["best_epoch"] = best_epoch if best_state is not None else epoch
48     return history

```

O laço roda até `epochs` vezes, atualizando os pesos a cada iteração e guardando *loss/accuracy* de treino em `history`. Quando há conjunto de validação, o método também mede o *loss* nele e compara com o melhor já visto (`best_val_loss`): se melhorou, guarda uma cópia dos pesos daquela época (`best_state`) e zera o contador de estagnação (`epochs_without_improvement`); se não, incrementa esse contador. Quando o contador atinge `patience` (25 épocas nesta busca), o `break` interrompe o laço antes de completar todas as `epochs`. No final, os pesos do modelo são restaurados para `best_state`, não para os da última época, e `history["best_epoch"]` guarda em qual época isso aconteceu, valor reaproveitado

depois no treino final (Subseção 4.5.5). Sem essa restauração, o modelo ficaria com os pesos da última época mesmo quando ela já estivesse pior que uma época anterior, ou seja, já sobreajustada ao treino.

4.5.4 Busca de hiperparâmetros

Três hiperparâmetros descobri durante a busca: número de neurônios na camada escondida, taxa de aprendizado e peso da regularização L2. Escolhi cada um por busca em grade (18 combinações: $\{32, 48, 64\} \times \{0,05, 0,1\} \times \{0, 0,001, 0,01\}$), usando F1-macro **na validação** (nunca no teste, para não invalidar seu uso como medida honesta de desempenho em dado novo) e a mesma parada antecipada descrita acima (*patience* de 25 épocas).

A combinação com melhor desempenho foi `hidden_size=48`, `learning_rate=0,1` e `l2_lambda=0,001` (F1-macro de validação de 0,674), uma entre seis das dez melhores combinações que usaram o orçamento inteiro de 500 épocas sem acionar a parada antecipada. Nem toda combinação precisou do orçamento inteiro: a mais rápida a convergir entre as dez melhores parou em 292 épocas (`hidden_size=32`, `learning_rate=0,1`, sem regularização L2), o que mostra que a parada antecipada estava de fato ativa e cortando o treino quando fazia sentido, não apenas rodando até o limite por padrão.

4.5.5 Treinamento final

Depois retreinei o modelo final combinando treino e validação (o hiperparâmetro já estava escolhido nesta etapa) pelas 500 épocas encontradas na busca, sem separar validação nem aplicar parada antecipada de novo. O *loss* de treino caiu de 3,574 na primeira época para 0,872 na última, com a *accuracy* de treino subindo de 0,05 para 0,73 no mesmo intervalo. A curva desce de forma suave, sem oscilação; se a taxa de aprendizado estivesse alta demais, seria esperado um padrão serrilhado (o gradiente ultrapassando o mínimo repetidamente a cada passo), então essa suavidade é mais um indício de que `learning_rate=0,1` foi uma escolha adequada para esse problema, e não só o valor que venceu a busca em validação.

4.6 Resultados e métricas

A Tabela 3 resume o desempenho no conjunto de teste (sujeitos nunca vistos em treino ou validação). O `MLPHardcode` supera o *baseline* com folga (acurácia de 0,676 contra 0,056). Para reduzir o risco de um erro sutil na implementação manual (por exemplo, uma derivada errada que ainda assim converge, só com desempenho abaixo do esperado), treinei também um segundo modelo com biblioteca, configurado com hiperparâmetros equivalentes; a tabela e sua nota trazem os detalhes dessa validação. Os dois ficam próximos entre si, o que não é uma prova formal de que a implementação manual está correta, mas é evidência forte: duas implementações independentes convergindo para resultados semelhantes dificilmente é coincidência.

A diferença residual entre as duas implementações do MLP (cerca de 0,2 ponto percentual) provavelmente vem de detalhes difíceis de controlar entre

bibliotecas, como geradores de números aleatórios distintos para a inicialização mesmo com a mesma *seed*, e não de um erro de implementação.

Tabela 3 – *Comparação de desempenho no conjunto de teste (18 classes).*

Modelo	Acurácia	Precisão (macro)	Recall (macro)	F1 (macro)
Baseline majoritário	0,056	0,003	0,056	0,006
MLP (implementado do zero)	0,676	0,672	0,676	0,668
MLP (scikit-learn)	0,678	0,675	0,679	0,670

Fonte: Elaborada pelo autor, a partir de `ml/mlp_activity_classification.ipynb` (seed=42) e `ml/export_model_results.py`.

4.7 Avaliação qualitativa

A acurácia agregada esconde informação: o desempenho por classe (Tabela 4) mostra que as cinco piores classes por F1 (comer sopa, comer batata frita, comer macarrão, beber e comer sanduíche) não são um grupo aleatório, mas variações do mesmo gesto (levar a mão à boca, sentado, com pouco movimento), entre 54,9% e 89,2% do erro de cada uma delas vai para outra classe do mesmo grupo, e 27,5% de todo o erro do modelo é interno a esse grupo, indício de que o sensor de pulso, isoladamente, não contém informação suficiente para diferenciar gestos que diferem principalmente pelo objeto na mão, não pelo movimento em si. Testei essa hipótese diretamente fundindo as cinco classes em uma única categoria (18 → 14 classes, com nova busca de hiperparâmetros): a acurácia sobe de 0,676 para 0,777, confirmando a causa física, e não um limite do modelo, ao custo de perder a distinção entre os tipos específicos de refeição, uma troca entre granularidade e confiabilidade registrada aqui como decisão consciente.

4.8 Limitações do modelo

- apenas o dispositivo *watch* foi usado; o *dataset* também tem dados de *phone*, deixados de fora nesta primeira versão, o que restringe a aplicação a usuários de *smartwatch* compatível;
- 51 sujeitos é uma amostra pequena para generalizar para a população em geral, embora suficiente para o escopo de disciplina deste projeto;
- as cinco atividades de comer/beber sentado (Seção 4.7) não são bem distinguidas apenas com sensores de pulso, uma limitação física dos dados de entrada, não do algoritmo;

- a magnitude do vetor de médias usada na camada *gold* (Seção 3.3) é uma aproximação simples de intensidade de movimento, não a média da magnitude instantânea (que exigiria o sinal bruto, indisponível na camada *gold*).

Tabela 4 – *Relatório de classificação por atividade: melhores classes, grupo comer/beber e pior caso (conjunto de teste, MLP implementado do zero).*

Código	Atividade	Precisão	Recall	F1	Suporte
B	correr (<i>jogging</i>)	0,89	0,97	0,93	139
R	bater palmas	0,95	0,89	0,92	142
S	dobrar roupa	0,82	0,83	0,83	143
H	comer sopa	0,64	0,79	0,71	141
I	comer batata frita	0,59	0,61	0,60	140
J	comer macarrão	0,62	0,54	0,58	154
K	beber	0,53	0,65	0,58	142
L	comer sanduíche	0,27	0,14	0,18	139

Fonte: Elaborada pelo autor, a partir de `ml/export_model_results.py`; relatório completo (18 classes) disponível no repositório.

5 VISUALIZAÇÃO DA INFORMAÇÃO E REPRODUTIBILIDADE

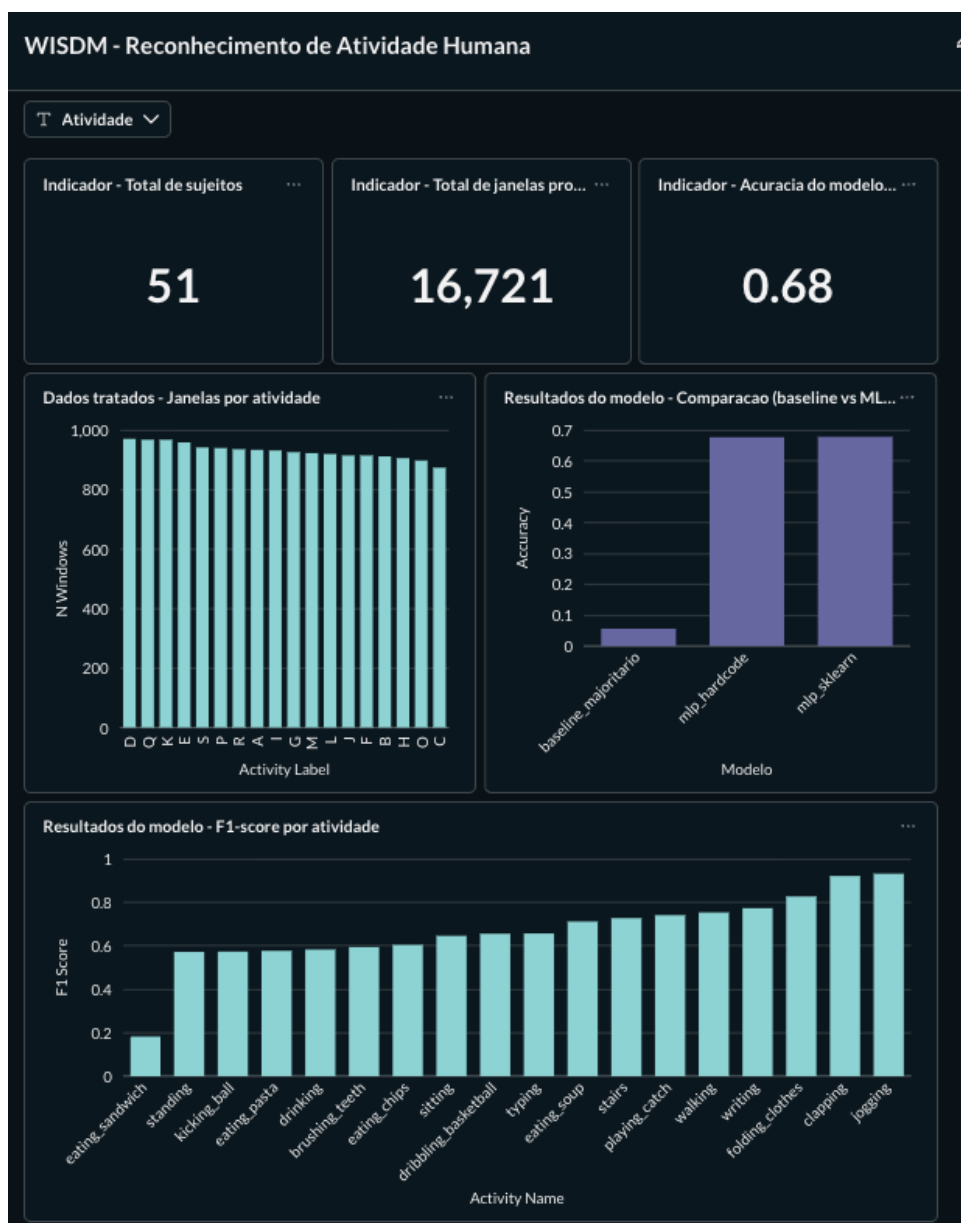
5.1 Painel de visualização

Os resultados estão disponíveis em um painel no Metabase (METABASE, 2026), ferramenta de BI que conecta direto num banco/*data warehouse* e monta gráficos e indicadores sem precisar programar cada visualização. Ele foi executado em contêiner (*Docker*) e fica conectado ao banco *gold* via *Athena*, consumindo diretamente as tabelas da camada ouro (Seção 3.3) e as tabelas de resultados do modelo publicadas por `ml/export_model_results.py` (Seção 4.6), sem transformação adicional além do que o *dbt* já faz.

A Figura 7 mostra o painel *WISDM - Reconhecimento de Atividade Humana*. Ele reúne três indicadores principais (51 sujeitos, 16.721 janelas processadas e 0,68 de acurácia do modelo, o mesmo valor da Tabela 3), a visualização dos dados tratados (janelas válidas por atividade) e a comparação de desempenho entre o *baseline* e os dois MLPs. O gráfico de F1-score por atividade, em ordem crescente,

reproduz visualmente o agrupamento das cinco atividades de comer/beber identificado na Seção 4.7. Um filtro por *Atividade*, no topo do painel, permite o recorte analítico exigido pelo enunciado.

Figura 7 – *Painel Metabase WISDM - Reconhecimento de Atividade Humana: indicadores, dados tratados e resultados do modelo.*



Fonte: Elaborada pelo autor.

A ideia do dashboard é mostrar o panorama geral do dados e do modelo, com indicadores principais, dados tratados e resultados do modelo, e permitir o recorte analítico por atividade. O painel apoia diretamente a decisão descrita no Seção 1.2: dá para comparar o desempenho do modelo entre atividades e identificar onde a solução é confiável o suficiente para apoiar a decisão do aplicativo, e onde (como no grupo de comer/beber do Seção 4.7).

5.2 Reprodutibilidade do repositório

Versionei todo o código, configuração e instruções necessários para reproduzir as etapas do *pipeline* no repositório do projeto¹, organizado por responsabilidade:

- airflow/** DAGs (Capítulo 3), pacote `wisdm_silver` com a lógica de extração de atributos e utilitários de Athena/Glue, e capturas de tela de execução;
- dbt/** projeto dbt completo (*staging*, *marts*, *gold*), com documentação e testes (Seção 3.3);
- infra/** template CloudFormation e script de *deploy* da infraestrutura AWS (Seção 2.3);
- arquitetura/** diagrama da arquitetura implementada e registro de custos (Capítulo 2);
- ml/** notebook de treino do modelo, script de publicação de resultados e um *cache* local em Parquet de `fct_wisdm_windows`, para permitir rodar o notebook sem depender de uma sessão AWS ativa (Capítulo 4);
- metabase/** provisionamento do painel de visualização (Seção 5.1).

A execução ponta a ponta acontece com *Docker Compose* (`docker-compose.yaml`), a partir de um arquivo `.env` (modelo em `.env.example`) com as credenciais AWS e Kaggle necessárias. Provisiono a infraestrutura AWS uma única vez com `infra/deploy_infra.sh`, antes da primeira execução das DAGs, e depois as três DAGs do Capítulo 3 podem ser disparadas manualmente ou deixadas em execução automática pelo encadeamento por *Airflow Assets* já descrito. As evidências de execução usadas neste relatório vêm diretamente dessa execução real do *pipeline*.

6 CONCLUSÃO

Esse trabalho foi desafiador e gratificante onde eu tiver a oportunidade de contruir um *pipeline* de dados em nuvem completo, da ingestão de sinais brutos de *smartwatch* até um painel de visualização voltado à decisão, cobrindo os requisitos das três disciplinas integradas no Módulo 2. Processo os dados brutos e semiestruturados do WISDM (Seção 2.1) numa arquitetura medalhão orquestrada pelo Airflow, com extração de atributos e sincronização de sensores (Seção 3.2), modelagem em camadas com dbt sobre Amazon Athena (Seção 3.3) e infraestrutura como código via CloudFormation (Seção 2.3). Resolvi a tarefa de Aprendizagem de Máquina, classificação de 18 atividades a partir de atributos de acelerômetro e giroscópio, com um MLP implementado do zero e validado contra o `MLPClassifier` do *scikit-learn* (Capítulo 4), superando com folga um *baseline* majoritário e atingindo acurácia e F1-macro próximos de 0,68. Os resultados alimentam um painel de visualização (Seção 5.1) que apoia a decisão original: identificar automaticamente a atividade de um usuário a partir de dados de pulso.

¹<https://github.com/coutinhonobre/Projeto-Integrado-Segundo-Bimestre>

6.1 Limitações

Além das limitações específicas do modelo já discutidas no Seção 4.8, a solução como um todo tem limitações de escopo assumidas para caber no escopo de disciplina:

- orquestração, treino do modelo e painel rodam localmente via *Docker Compose*, não continuamente em nuvem, decisão dada o ambiente de laboratório disponível (Seção 2.4);
- Glue Data Catalog e Athena substituem o Snowflake sugerido no enunciado, decisão de escopo autorizada previamente para o ambiente de *sandbox*;
- processo apenas o dispositivo *watch*; os dados de *phone*, embora armazenados na bronze, ainda não são consumidos pelo *pipeline*;
- o registro de custos (Seção 2.5) é uma estimativa via *AWS Pricing Calculator*, não uma fatura real, já que a conta de laboratório não expõe Cost Explorer/-Billing.

6.2 Próximos passos

Como evolução natural da solução, deixo registrado: migrar a orquestração, o treino e o painel para os serviços gerenciados propostos no Seção 2.4 (MWAA, SageMaker, ECS Fargate/RDS); processar também os sinais do *phone*, avaliando se a fusão com os dados de pulso melhora a distinção das atividades de comer/beber (Seção 4.7); ampliar a amostra de sujeitos para avaliar a generalização fora do *dataset* WISDM; ampliar o painel (Seção 5.1) com matriz de confusão completa e recorte por sujeito; e avaliar um classificador hierárquico para as classes de comer/beber (decidindo primeiro entre “sentado comendo/bebendo” e as demais atividades, só depois o item específico), em vez da fusão definitiva de classes usada neste projeto. Tenho interesse, também, em explorar a viabilidade de embarcar esse tipo de modelo diretamente no *smartphone* ou no *smartwatch* (inferência *on-device*), eliminando a dependência de conectividade e o custo de nuvem por predição; esse caminho exigiria a validação de desempenho dentro das restrições de memória, processamento e bateria de um dispositivo móvel.

Referências

AMAZON WEB SERVICES. **Amazon Web Services (AWS)**. 2026. Disponível em: <https://aws.amazon.com/>. Acesso em: 13 ago 2026. 3, 6

APACHE SOFTWARE FOUNDATION. **Apache Airflow Documentation**. 2024. Disponível em: <https://airflow.apache.org/docs/>. Acesso em: 12 ago 2026. 8

DBT LABS. **dbt Documentation**. 2024. Disponível em: <https://docs.getdbt.com/>. Acesso em: 12 ago 2026. 11

HARRIS, Charles R.; MILLMAN, K. Jarrod; WALT, Stéfan J. van der; GOMMERS, Ralf; VIRTANEN, Pauli; COURNAPEAU, David; WIESER, Eric; TAYLOR, Julian; BERG, Sebastian; SMITH, Nathaniel J.; KERN, Robert; PICUS, Matti; HOYER, Stephan; KERKWIJK, Marten H. van; BRETT, Matthew; HALDANE, Allan; RÍO, Jaime Fernández del; WIEBE, Mark; PETERSON, Pearu; GÉRARD-MARCHANT, Pierre; SHEPPARD, Kevin; REDDY, Tyler; WECKESSER, Warren; ABBASI, Hameer; GOHLKE, Christoph; OLIPHANT, Travis E. Array programming with NumPy. **Nature**, v. 585, n. 7825, p. 357–362, 2020. 15

KIMBALL, Ralph; ROSS, Margy. **The Data Warehouse Toolkit: The Definitive Guide to Dimensional Modeling**. 3. ed. Indianapolis: Wiley, 2013. 11

METABASE. **Metabase**. 2026. Disponível em: <https://www.metabase.com/>. Acesso em: 13 ago 2026. 20

NOBRE, Igor Coutinho. **Projeto Integrado – Segundo Bimestre**. 2026. Repositório GitHub. Disponível em: <https://github.com/coutinhonobre/Projeto-Integrado-Segundo-Bimestre>. Acesso em: 13 ago 2026. 7, 9, 11, 15, 16

WEISS, Gary. **WISDM Smartphone and Smartwatch Activity and Biometrics Dataset**. [S.l.]: UCI Machine Learning Repository, 2019. Dataset. Acesso em: 12 ago 2026. 3, 4, 5